과제2 · 이벤트 드리븐 아키텍처 스타일 시스템 구현

온라인 경매 시스템 — 브로커 토폴로지 (Python asyncio)

과목	소프트웨어아키텍처설계
학번	202110914
이름	김대진
제출일	2026년 10월 25일

1. 사용한 프롬프트

Claude Code(Claude Opus 5.5)에 아래 두 프롬프트를 주어 설계·구현했습니다. 프롬프트 2의 구현은 Claude Code가 띄운 Sonnet 서브에이전트가 맡았고, 결과 코드를 검토해 로그 문구 한 곳을 고쳤습니다.

프롬프트 1 — 과제 전달

> 이벤트 드리븐 아키텍처 스타일을 가지는 시스템 구현
> (시스템의 규모보다 이벤트 드리븐 아키텍처 스타일의 컴포넌트, 커넥터, 제약조건을 정확히 따르는 것이 중요함)
> 제출 내용: 1. 사용한 프롬프트 2. 시스템 주요 구성 컴포넌트를 포함한 전체 동작 흐름 3. 실행 결과 캡쳐 4. 기타 시스템 설명에 필요한 내용

프롬프트 2 — 구현 지시

```
Python 3 표준 라이브러리(asyncio)만 써서, "브로커 토폴로지 이벤트 드리븐 아키텍처"를 따르는
온라인 경매 시스템을 하나의 실행 가능한 프로그램으로 구현해 줘. 규모는 작아도 되지만
이벤트 드리븐 스타일의 컴포넌트·커넥터·제약조건을 정확히 지켜야 한다.

[도메인]
입찰자가 물건에 입찰한다. 입찰 프로듀서가 만든 입찰 이벤트를 입찰 캡처, 입찰 추적,
입찰 분석 서비스가 각자 받아 처리한다. 경매가 끝나면 낙찰 처리를 한다.

[컴포넌트]
1. 이벤트 브로커(EventBroker): 이벤트를 "전달만" 한다. 워크플로우 판단·비즈니스 로직 금지.
   - 이벤트 채널 = 토픽. 한 브로커에 여러 토픽(bid.placed, bid.highest_changed,
     auction.closed, auction.settled, notification.sent, deadletter)을 둔다.
   - 구독(subscription)마다 자기 큐를 가진다. 같은 토픽의 구독자들은 서로 간섭 없이
     각자 속도로 소비한다(발행/구독).
   - 같은 구독 이름으로 워커를 여러 개 붙이면 한 큐를 나눠 소비한다(경쟁 소비자 = 그 프로세서만 확장).
2. 이벤트 발행자(시작 이벤트): BidProducer(입찰자 입찰 → BidPlaced), AuctionClock(마감 → AuctionClosed).
3. 이벤트 구독자(이벤트 프로세서): 처리 후 "처리 이벤트"를 발행할 수 있다.
   - BidCapture: 모든 입찰을 기록한다.
   - BidTracker: 최고가를 추적하고, 최고가가 바뀌면 HighestBidChanged를 발행한다.
   - BidAnalytics: 느린 통계 처리(입찰 수, 평균가). 워커 2개로 확장한다.
   - Notifier: HighestBidChanged·AuctionSettled를 받아 알림을 보내고 NotificationSent를 발행한다.
   - Settlement: AuctionClosed를 받아 낙찰자를 확정하고 AuctionSettled를 발행한다.
     (최고가 정보는 BidTracker를 호출하지 말고 자신이 구독한 이벤트로 스스로 유지한다.)

[커넥터]
- 통신 메커니즘: 브로커의 publish/subscribe와 구독별 asyncio.Queue.
- 데이터 형식 변환(어댑터): 이벤트는 JSON 문자열로 직렬화해 채널에 넣고, 꺼낼 때 역직렬화한다.
- 이벤트 봉투(envelope): event_id, type(과거형 이름), topic, timestamp, ttl, payload.

[제약조건 — 반드시 지키고 코드로 검사]
C1. 컴포넌트끼리 직접 참조·호출하지 않는다. 브로커를 통해서만 통신한다.
    실행 시작 시 각 컴포넌트의 속성을 검사해 다른 컴포넌트 참조가 없음을 확인하고 결과를 출력한다.
C2. 비동기 통신: publish는 기다리지 않고 즉시 반환한다. 발행자는 처리 결과를 기다리지 않는다.
    입찰 발행에 걸린 시간과 분석 처리에 걸린 시간을 비교해 출력한다.
C3. 이벤트는 이미 일어난 사실(과거형)이고 응답을 요구하지 않는다.
```

```
C4. 익명성: 발행자는 구독자를 모른다(토픽만 안다).

[메시지 전달 신뢰성]
- 승인(ack): 구독자가 처리를 끝내면 ack한다. 예외가 나면 최대 3번 재시도하고,
  그래도 실패하면 deadletter 토픽으로 보낸다.
- 만료(TTL): 만료된 이벤트는 처리하지 않고 버린 뒤 로그를 남긴다.
- 내고장성 시연: BidAnalytics가 특정 입찰(금액이 13의 배수)에서 일부러 실패하게 하고,
  다른 구독자는 영향 없이 계속 처리됨을 보인다.

[실행·출력]
- `python auction_eda.py` 한 번으로 끝까지 돌고 5초 안에 종료한다. random seed 고정.
- 콘솔에 시각(경과 ms)·컴포넌트·이벤트를 한 줄씩 한국어로 출력한다. UTF-8로 출력한다.
- 마지막에 요약(토픽별 발행 수, 구독별 처리·재시도·DLQ·만료 수, 낙찰 결과, 제약 검사 결과)을 표로 출력한다.
- 같은 내용을 run_log.txt로도 저장한다.
- 파일은 auction_eda.py 하나. 클래스·메서드에 짧은 한국어 주석을 단다.
```

2. 시스템 구성과 전체 동작 흐름

2.1 도메인과 스타일 선택 이유

온라인 경매는 입찰이 동시에 많이 들어옵니다. 입찰 하나를 기록·추적·분석·알림 서비스가 각자 처리해야 하므로 **응답성·확장성·기능확장성**이 중요합니다. 그래서 발행자와 구독자를 떼어 놓는 이벤트 드리븐 스타일을 골랐습니다.

흐름이 단순하고 처리 서비스끼리 순서를 맞출 필요가 거의 없으므로 중재자 대신 **브로커 토폴로지**를 썼습니다. 브로커는 워크플로우를 제어하지 않고 이벤트를 전달만 합니다.

2.2 컴포넌트 · 커넥터 · 제약조건

구분	스타일의 요소	이 시스템의 구현
컴포넌트	이벤트 발행자 (시작 이벤트)	`BidProducer` (입찰 → BidPlaced), `AuctionClock` (마감 → AuctionClosed)
	이벤트 브로커	`EventBroker` : 토픽 6개와 구독별 큐를 관리합니다. 비즈니스 로직은 없습니다.
	이벤트 구독자 (처리 후 처리 이벤트 발행 가능)	`BidCapture` , `BidTracker` , `BidAnalytics` (워커 2개), `Notifier` , `Settlement` , `DeadLetterMonitor`
커넥터	이벤트 채널	토픽 6개: `bid.placed` , `bid.highest_changed` , `auction.closed` , `auction.settled` , `notification.sent` , `deadletter`
	통신 메커니즘	`publish()` / `subscribe()` , 구독마다 하나씩 있는 `asyncio.Queue`
	데이터 형식 변환 (어댑터)	`Envelope.dumps()` / `loads()` : 이벤트를 JSON 문자열로 바꿔 채널에 넣고, 꺼낼 때 객체로 되돌립니다. 봉투 필드는 event_id, type, topic, timestamp, ttl, payload입니다.
제약조건	C1 컴포넌트끼리 직접 참조·호출하지 않음	모든 컴포넌트는 `broker` 만 가집니다. 실행 시작 때 속성을 검사합니다.
	C2 비동기 통신	`publish()` 는 `put_nowait` 로 바로 반환합니다. 발행자는 처리 결과를 기다리지 않습니다.
	C3 이벤트 = 이미 일어난 사실, 응답을 요구하지 않음	이벤트 이름이 모두 과거형입니다(BidPlaced, AuctionClosed ...). 반환값을 쓰지 않습니다.
	C4 익명성 (발행자는 구독자를 모름)	발행자는 토픽 이름만 압니다. 구독자 목록은 브로커 안에만 있습니다.

2.3 구성도

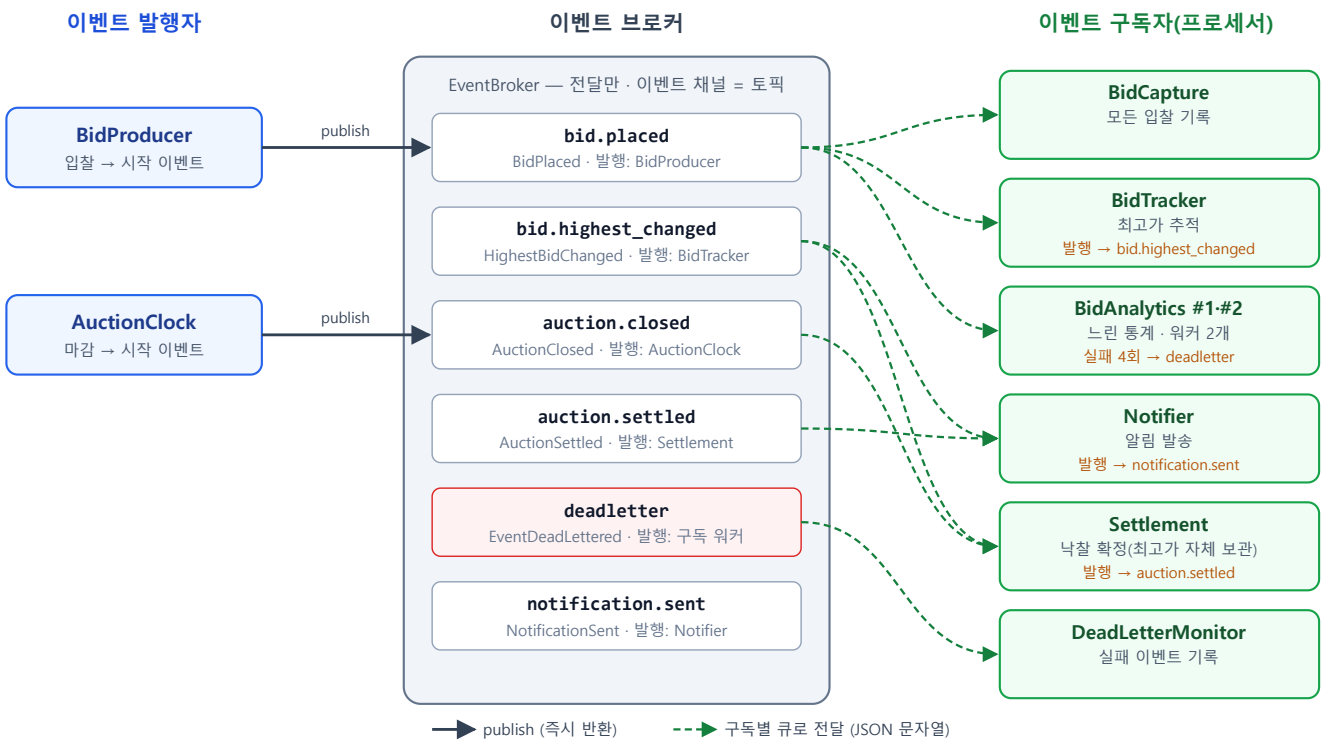


그림 1. 컴포넌트와 커넥터. 실선은 발행, 점선은 브로커가 구독별 큐로 전달하는 경로입니다. 구독자가 발행하는 처리 이벤트는 상자 아래 주황 글씨로 표시했습니다(선을 그리면 교차가 많아 생략).

2.4 이벤트 목록

토픽	이벤트	발행자	구독자
`bid.placed`	BidPlaced	BidProducer	BidCapture, BidTracker, BidAnalytics
`bid.highest_changed`	HighestBidChanged	BidTracker	Notifier, Settlement
`auction.closed`	AuctionClosed	AuctionClock	Settlement
`auction.settled`	AuctionSettled	Settlement	Notifier
`notification.sent`	NotificationSent	Notifier	(없음 — 외부 연계용으로 열어 둠)
`deadletter`	EventDeadLettered	구독 워커(재시도 초과)	DeadLetterMonitor

2.5 전체 동작 흐름

괄호 안 시각은 3장 실행 결과의 경과 시간입니다.

1. **시작·제약 검사**: 브로커와 컴포넌트 8개를 만들고, 구독자는 생성 시 토픽을 구독합니다. 각 컴포넌트에 다른 컴포넌트 참조가 없는지 검사합니다(C1).
2. **입찰 발행**: `BidProducer`가 입찰 26건을 `bid.placed`에 발행합니다. 발행 전체가 0.344ms 만에 끝나고, 처리 결과는 기다리지 않습니다(2.1ms).

3. **브로커 전달**: 브로커는 이벤트를 JSON으로 바꿔 `bid.placed`의 구독 큐 3개(Capture·Tracker·Analytics)에 각각 넣습니다. 구독자는 자기 큐를 자기 속도로 소비합니다.
4. **기록·추적**: `BidCapture`가 입찰을 기록합니다. `BidTracker`는 최고가가 바뀔 때마다 처리 이벤트 `HighestBidChanged`를 발행합니다(317원 → 392원 → 398원).
5. **연쇄 반응**: `HighestBidChanged`를 구독한 `Notifier`가 알림을 보냅니다. `Settlement`는 BidTracker를 호출하지 않고 이 이벤트로 자기 최고가를 유지합니다. 마지막 알림 시각은 3.2ms입니다.
6. **느린 분석·장애 격리**: `BidAnalytics`는 워커 2개가 한 큐를 나눠 처리합니다. 13의 배수 입찰에서 일부러 실패하면 3번 재시도한 뒤 `deadletter`로 보내고(첫 DLQ 235.1ms), `DeadLetterMonitor`가 받아 기록합니다. 다른 구독자는 이미 처리를 끝냈습니다.
7. **만료**: ttl=0으로 보낸 입찰 3건은 모든 구독자가 처리하지 않고 버립니다.
8. **마감·낙찰**: `AuctionClock`이 `AuctionClosed`를 발행합니다(855.0ms). `Settlement`가 낙찰자를 확정해 `AuctionSettled`를 발행하고, `Notifier`가 결과를 알립니다(855.2ms).
9. **요약**: C2~C4를 검사하고, 토픽별 발행 수와 구독별 처리·재시도·DLQ·만료 수를 출력합니다.

3. 실행 결과 캡처

`python auction_eda.py`를 한 번 실행한 콘솔 출력입니다(Python 3.14, Windows 11, 약 1초). 출력 138줄 전체를 세 장으로 나눠 실었습니다. 화면은 콘솔 출력을 터미널 화면으로 그려 캡처한 것이고, 같은 내용이 `run_log.txt`에 저장됩니다.

python auction_eda.py — 실행 결과

```
PS C:\...\과제2-이벤트드리븐> python auction_eda.py
[     1.6ms] 시스템               | 경매 시스템 시작
[     1.6ms] 제약검사C1           | BidProducer       속성 6개 검사, 타 컴포넌트 참조 0, 브로커 참조 있음 -> OK
[     1.6ms] 제약검사C1           | AuctionClock      속성 5개 검사, 타 컴포넌트 참조 0, 브로커 참조 있음 -> OK
[     1.6ms] 제약검사C1           | BidCapture        속성 6개 검사, 타 컴포넌트 참조 0, 브로커 참조 있음 -> OK
[     1.7ms] 제약검사C1           | BidTracker        속성 6개 검사, 타 컴포넌트 참조 0, 브로커 참조 있음 -> OK
[     1.7ms] 제약검사C1           | BidAnalytics      속성 10개 검사, 타 컴포넌트 참조 0, 브로커 참조 있음 -> OK
[     1.7ms] 제약검사C1           | Notifier          속성 6개 검사, 타 컴포넌트 참조 0, 브로커 참조 있음 -> OK
[     1.7ms] 제약검사C1           | Settlement        속성 7개 검사, 타 컴포넌트 참조 0, 브로커 참조 있음 -> OK
[     1.7ms] 제약검사C1           | DeadLetterMonitor 속성 6개 검사, 타 컴포넌트 참조 0, 브로커 참조 있음 -> OK
[     1.7ms] 제약검사C1           | 검사기 자가검증(A가 B를 참조하게 만든 가짜 객체) -> 위반 탐지 성공
[     1.7ms] 제약검사C1           | 결과: PASS
[     2.1ms] BidProducer          | 입찰 26건 발행 완료 (0.344ms, 처리 결과는 기다리지 않음)
[     2.3ms] BidCapture           | 입찰 기록 영희 317원
[     2.3ms] BidCapture           | 입찰 기록 영희 294원
[     2.3ms] BidCapture           | 입찰 기록 지영 215원
[     2.3ms] BidCapture           | 만료로 폐기 evt-0004 BidPlaced (ttl=0ms)
[     2.3ms] BidCapture           | 입찰 기록 민수 392원
[     2.4ms] BidCapture           | 입찰 기록 민수 327원
[     2.4ms] BidCapture           | 입찰 기록 철수 143원
[     2.4ms] BidCapture           | 입찰 기록 지영 309원
[     2.4ms] BidCapture           | 입찰 기록 영희 286원
[     2.4ms] BidCapture           | 입찰 기록 하준 173원
[     2.4ms] BidCapture           | 입찰 기록 영희 169원
[     2.4ms] BidCapture           | 만료로 폐기 evt-0012 BidPlaced (ttl=0ms)
[     2.4ms] BidCapture           | 입찰 기록 하준 273원
[     2.4ms] BidCapture           | 입찰 기록 철수 325원
[     2.4ms] BidCapture           | 입찰 기록 지영 299원
[     2.5ms] BidCapture           | 입찰 기록 지영 143원
[     2.5ms] BidCapture           | 입찰 기록 영희 283원
[     2.5ms] BidCapture           | 입찰 기록 영희 398원
[     2.5ms] BidCapture           | 입찰 기록 민수 142원
[     2.5ms] BidCapture           | 입찰 기록 철수 347원
[     2.5ms] BidCapture           | 만료로 폐기 evt-0021 BidPlaced (ttl=0ms)
[     2.5ms] BidCapture           | 입찰 기록 지영 122원
[     2.6ms] BidCapture           | 입찰 기록 하준 131원
[     2.7ms] BidCapture           | 입찰 기록 민수 151원
[     2.7ms] BidCapture           | 입찰 기록 철수 116원
[     2.8ms] BidCapture           | 입찰 기록 지영 231원
[     2.8ms] BidTracker           | 최고가 갱신 영희 317원 -> HighestBidChanged 발행
[     2.9ms] BidTracker           | 만료로 폐기 evt-0004 BidPlaced (ttl=0ms)
[     2.9ms] BidTracker           | 최고가 갱신 민수 392원 -> HighestBidChanged 발행
[     3.0ms] BidTracker           | 만료로 폐기 evt-0012 BidPlaced (ttl=0ms)
[     3.0ms] BidTracker           | 최고가 갱신 영희 398원 -> HighestBidChanged 발행
[     3.1ms] BidTracker           | 만료로 폐기 evt-0021 BidPlaced (ttl=0ms)
[     3.2ms] Notifier             | 알림 발송: 새 최고가 317원 (영희)
[     3.2ms] Notifier             | 알림 발송: 새 최고가 392원 (민수)
[     3.2ms] Notifier             | 알림 발송: 새 최고가 398원 (영희)
```

그림 2. 출력 1~47줄 — C1 제약 검사(PASS), 입찰 26건 발행(0.344ms), BidCapture·BidTracker 처리, 최고가 변경 알림, 만료 폐기

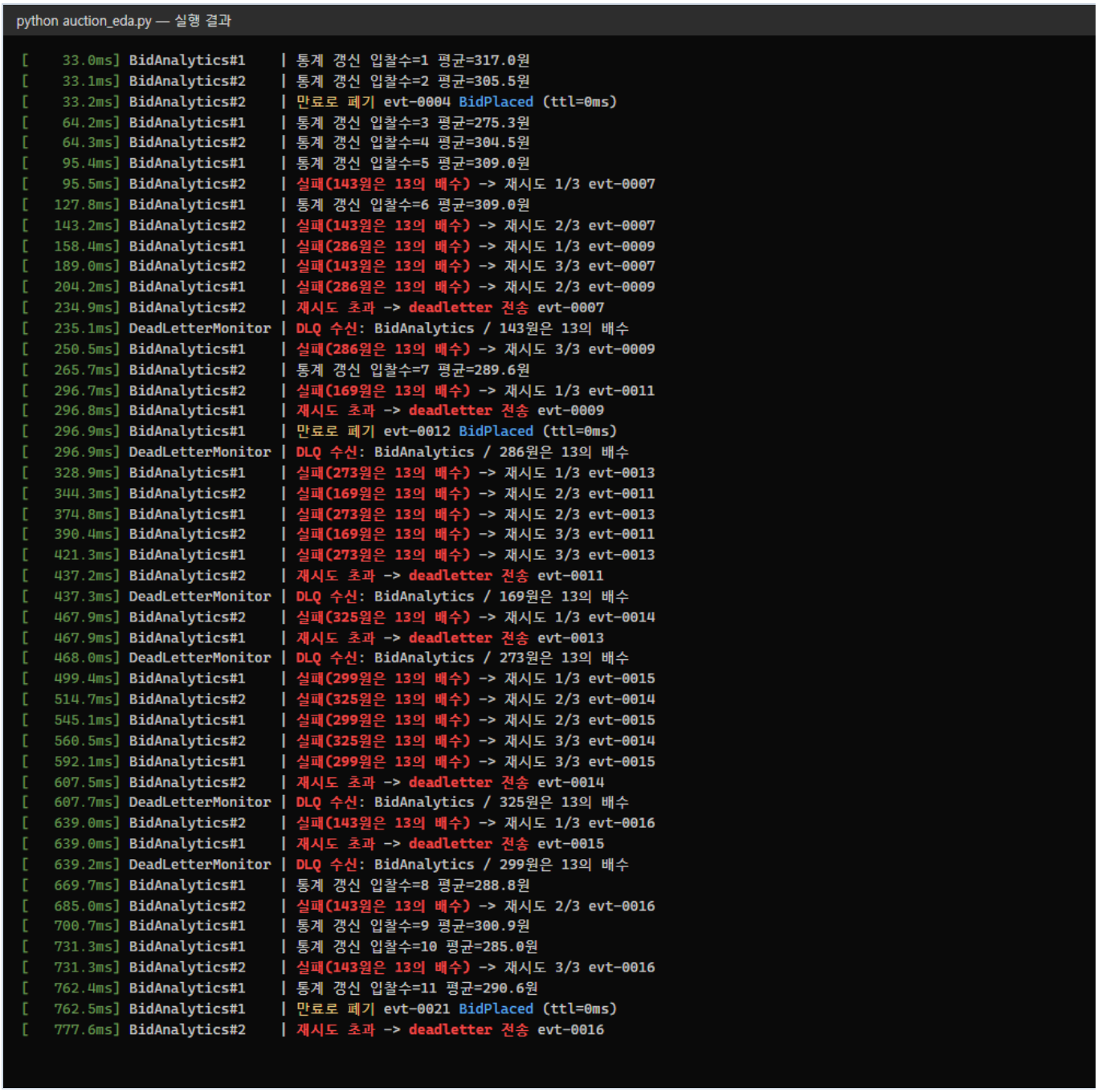


그림 3. 출력 48~95줄 — BidAnalytics 워커 #1·#2 분산 처리, 실패 → 재시도 1/3~3/3 → deadletter → DeadLetterMonitor 수신

python auction_eda.py — 실행 결과

```
[   777.8ms] DeadLetterMonitor | DLQ 수신: BidAnalytics / 143원은 13의 배수
[   793.1ms] BidAnalytics#1    | 통계 갱신 입찰수=12 평균=276.6원
[   808.6ms] BidAnalytics#2    | 통계 갱신 입찰수=13 평균=265.4원
[   823.8ms] BidAnalytics#1    | 통계 갱신 입찰수=14 평균=257.2원
[   839.1ms] BidAnalytics#2    | 통계 갱신 입찰수=15 평균=247.8원
[   854.8ms] BidAnalytics#1    | 통계 갱신 입찰수=16 평균=246.8원
[   855.0ms] AuctionClock      | A-100 경매 마감 -> AuctionClosed 발행
[   855.1ms] Settlement        | 낙찰 확정 영희 398원 -> AuctionSettled 발행
[   855.2ms] Notifier          | 알림 발송: 낙찰 결과: 영희 님 398원
[   855.3ms] 제약검사C2          | 입찰 전체 발행 0.344ms vs BidAnalytics 처리 851.7ms (약 2473배) -> 발행자는 처리 완료를 기다리지
않음
[   855.3ms] 제약검사C3          | 이벤트 type ['AuctionClosed', 'AuctionSettled', 'BidPlaced', 'EventDeadLettered',
'HighestBidChanged', 'NotificationSent'] -> PASS
[   855.3ms] 제약검사C4          | 발행자는 토픽 이름만 사용, 구독자 목록 미보유 -> PASS

======================================================================
요약
======================================================================
[토픽별 발행 수]
  bid.placed              26
  bid.highest_changed      3
  auction.closed           1
  auction.settled          1
  notification.sent        4
  deadletter               7

[구독별 처리/재시도/DLQ/만료]
  구독                        처리   재시도  DLQ   만료
  BidCapture                23      0    0    3
  BidTracker                23      0    0    3
  BidAnalytics              16     21    7    3
  Notifier/highest           3      0    0    0
  Settlement/highest         3      0    0    0
  Settlement/closed          1      0    0    0
  Notifier/settled           1      0    0    0
  DeadLetterMonitor          7      0    0    0
[BidAnalytics 워커별 성공 처리] {'BidAnalytics#1': 11, 'BidAnalytics#2': 5}
[낙찰 결과] A-100 낙찰자=영희 낙찰가=398원 (알림 4건 발송)
[BidTracker 최고가] 398원 / Settlement 보유 398원 -> 일치 True

[제약 검사]
  C1 직접 참조 없음    : PASS
  C2 비동기 발행       : 발행 0.344ms < 분석 851.7ms -> PASS
  C3 과거형 이벤트     : PASS
  C4 발행자 익명성     : PASS
```

그림 4. 출력 96~138줄 — 경매 마감, 낙찰 확정·알림, C2~C4 검사, 요약 표

4. 기타 — 설계 검증과 트레이드오프

4.1 제약조건을 지켰는지 확인한 방법

제약	확인 방법	결과
C1 직접 참조 없음	각 컴포넌트의 속성(`vars()`)을 재귀로 훑어 다른 컴포넌트 객체·바운드 메서드가 있는지 찾습니다. 검사기 자체가 동작하는지 보려고 일부러 A가 B를 참조하게 만든 객체로 위반을 잡아내는 자가검증도 합니다.	**PASS**
C2 비동기 발행	입찰 26건 발행 시간(0.344ms)과 BidAnalytics 처리 시간(851.7ms)을 비교합니다. 발행이 처리 완료를 기다렸다면 발행 시간이 처리 시간보다 길어야 합니다.	**PASS**
C3 과거형 이벤트	실제로 발행된 이벤트 type 6종이 모두 과거형인지 검사합니다.	**PASS**
C4 익명성	발행자 객체에 구독(Subscription) 정보가 없는지 검사합니다.	**PASS**

4.2 메시지 전달 신뢰성 (ED 슬라이드의 승인·만료)

- **승인(ack)**: 구독 워커는 처리가 끝나야 `task_done()`으로 승인합니다. 예외가 나면 최대 3번 재시도합니다.
- **데드레터**: 재시도를 넘기면 원래 이벤트를 `deadletter` 토픽으로 보내 잃지 않습니다. 실행 결과는 7건이었습니다.
- **만료(TTL)**: 봉투에 유효기간을 넣고, 만료된 이벤트는 처리하지 않고 버린 뒤 로그를 남깁니다.

4.3 실행에서 확인한 품질속성

품질속성	실행 결과에서 보이는 근거
응답성·성능	발행자는 0.344ms 만에 반환했습니다. 입찰자 알림은 3.2ms에 끝났고, 느린 분석(851.7ms)을 기다리지 않았습니다.
내고장성	BidAnalytics가 7건에서 실패하는 동안에도 BidCapture·BidTracker·Notifier는 23건을 모두 처리했습니다.
확장성	느린 BidAnalytics만 워커 2개로 늘렸고(#1 11건, #2 5건), 다른 서비스는 바꾸지 않았습니다.
기능확장성	새 구독자(예: DeadLetterMonitor)는 생성자에서 `listen()`으로 토픽을 구독하기만 하면 됩니다. 발행자 코드는 바뀌지 않습니다.

4.4 한계와 트레이드오프

- **흐름 제어가 없습니다.** 브로커는 "입찰 처리가 다 끝난 뒤 마감"을 보장하지 않습니다. 이 시연은 `broker.drain()`으로 큐가 빈 뒤에 마감 이벤트를 발행했습니다. 실제로는 마감 직전 입찰이

Settlement에 늦게 도착하면 낙찰자가 틀릴 수 있습니다(데이터 비일관성). 대책은 입찰 시각을 마감 시각과 비교하거나, 마감·낙찰 흐름만 중재자로 관리하는 것입니다.

- **복구성이 낮습니다.** deadletter로 간 분석 이벤트는 자동으로 다시 처리되지 않아 통계가 틀어집니다. 사람이 확인하거나 재투입 도구가 필요합니다.
- **디버깅이 어렵습니다.** 처리 순서가 구독자마다 달라서, 시각이 찍힌 로그 없이는 흐름을 따라가기 어렵습니다.
- **인메모리 브로커입니다.** 프로세스가 끝나면 큐가 사라집니다. 실제 시스템에서는 Kafka, RabbitMQ처럼 메시지를 영속 저장하는 브로커로 바꿉니다. 컴포넌트는 `publish/subscribe` 만 쓰므로 브로커만 교체하면 됩니다.

4.5 실행 방법과 파일

파일	내용
`auction_eda.py`	시스템 전체(표준 라이브러리만 사용, 401줄). `python auction_eda.py` 로 실행합니다.
`run_log.txt`	실행 출력 저장본
`prompt.md`	1장의 프롬프트 원문

부록. 소스 코드 (auction_eda.py)

```python
"""브로커 토폴로지 이벤트 드리븐 온라인 경매 시스템 (표준 라이브러리 asyncio만 사용)."""
import asyncio
import json
import os
import random
import sys
import time
from dataclasses import dataclass, asdict, field

MAX_RETRY = 3          # 처리 실패 시 최대 재시도 횟수
SEED = 20261025        # 난수 고정
T0 = time.perf_counter()
LOG: list[str] = []


def log(who: str, msg: str) -> None:
    """시각(경과 ms)·컴포넌트·내용을 한 줄로 출력하고 보관한다."""
    line = f"[{(time.perf_counter() - T0) * 1000:8.1f}ms] {who:<18}| {msg}"
    LOG.append(line)
    print(line)


def now_ms() -> float:
    return (time.perf_counter() - T0) * 1000


# ──────────────────────── 커넥터: 이벤트 봉투 + JSON 어댑터 ────────────────────────
@dataclass
class Envelope:
    """이벤트 봉투. 과거형 type, 토픽, 발생 시각(ms), 유효기간(ttl, ms), 내용(payload)."""
    event_id: str
    type: str
    topic: str
    timestamp: float
    ttl: float
    payload: dict = field(default_factory=dict)

    def dumps(self) -> str:      # 어댑터: 객체 -> JSON 문자열
        return json.dumps(asdict(self), ensure_ascii=False)

    @staticmethod
    def loads(raw: str) -> "Envelope":   # 어댑터: JSON 문자열 -> 객체
        return Envelope(**json.loads(raw))

    def expired(self) -> bool:
        return now_ms() - self.timestamp >= self.ttl


# ──────────────────────── 이벤트 브로커 ────────────────────────
class Subscription:
    """구독 하나 = 이름 + 자기 큐 + 통계. 같은 이름으로 붙은 워커들이 큐를 나눠 소비한다."""
    def __init__(self, name: str, topic: str):
        self.name, self.topic = name, topic
        self.queue: asyncio.Queue = asyncio.Queue()
        self.processed = self.retried = self.dlq = self.expired = 0


class EventBroker:
    """이벤트를 전달만 한다. 비즈니스 로직 없음. 토픽별 구독 큐에 JSON 문자열을 넣는다."""
    TOPICS = ["bid.placed", "bid.highest_changed", "auction.closed",
              "auction.settled", "notification.sent", "deadletter"]

    def __init__(self):
        self.subs: dict[str, list[Subscription]] = {t: [] for t in self.TOPICS}
```

```
        self.published = {t: 0 for t in self.TOPICS}
        self.seq = 0
        self.types_seen: set[str] = set()

    def subscribe(self, topic: str, name: str) -> Subscription:
        sub = Subscription(name, topic)
        self.subs[topic].append(sub)
        return sub

    def publish(self, topic: str, type_: str, payload: dict, ttl: float = 5000) -> None:
        """기다리지 않고 즉시 반환(put_nowait). 누가 구독하는지는 모른다."""
        self.seq += 1
        env = Envelope(f"evt-{self.seq:04d}", type_, topic, now_ms(), ttl, payload)
        raw = env.dumps()
        self.published[topic] += 1
        self.types_seen.add(type_)
        for sub in self.subs[topic]:
            sub.queue.put_nowait(raw)

    async def drain(self) -> None:
        """(시연 보조) 모든 큐가 비고 더 이상 새 발행이 없을 때까지 대기."""
        while True:
            before = self.seq
            await asyncio.gather(*(s.queue.join() for ss in self.subs.values() for s in ss))
            await asyncio.sleep(0)
            if self.seq == before:
                return

    def close(self, workers_per_sub: dict[str, int]) -> None:
        """워커 종료 신호(None)를 구독별 워커 수만큼 넣는다."""
        for ss in self.subs.values():
            for s in ss:
                for _ in range(workers_per_sub.get(s.name, 1)):
                    s.queue.put_nowait(None)


# ──────────────────────── 컴포넌트 공통 ────────────────────────
class Component:
    """모든 컴포넌트의 기반. 브로커만 알고 다른 컴포넌트는 모른다."""
    def __init__(self, name: str, broker: EventBroker):
        self.name = name
        self.broker = broker
        self.subs: list[Subscription] = []
        self.tasks: list[asyncio.Task] = []
        self.workers: dict[str, int] = {}

    def listen(self, topic: str, handler, workers: int = 1, sub_name: str | None = None) -> None:
        """토픽을 구독하고 워커를 띄운다. 워커가 여러 개면 한 큐를 나눠 소비(경쟁 소비자)."""
        sub = self.broker.subscribe(topic, sub_name or self.name)
        self.subs.append(sub)
        self.workers[sub.name] = workers
        for i in range(1, workers + 1):
            wid = f"{self.name}#{i}" if workers > 1 else self.name
            self.tasks.append(asyncio.create_task(self._worker(sub, handler, wid)))

    async def _worker(self, sub: Subscription, handler, wid: str) -> None:
        """큐에서 꺼내 역직렬화 -> TTL 검사 -> 처리 -> ack. 실패 시 재시도, 초과 시 deadletter."""
        while True:
            raw = await sub.queue.get()
            if raw is None:
                sub.queue.task_done()
                return
            env = Envelope.loads(raw)
            if env.expired():                       # 만료 이벤트는 처리 없이 버리고 로그
                sub.expired += 1
                log(wid, f"만료로 폐기 {env.event_id} {env.type} (ttl={env.ttl:g}ms)")
                sub.queue.task_done()
                continue
            for attempt in range(MAX_RETRY + 1):
                try:
                    await handler(env, wid)
```

```
                    sub.processed += 1              # ack
                    break
                except Exception as e:
                    if attempt < MAX_RETRY:
                        sub.retried += 1
                        log(wid, f"실패({e}) -> 재시도 {attempt + 1}/{MAX_RETRY} {env.event_id}")
                        await asyncio.sleep(0.006)
                    else:
                        sub.dlq += 1
                        log(wid, f"재시도 초과 -> deadletter 전송 {env.event_id}")
                        self.broker.publish("deadletter", "EventDeadLettered",
                                            {"subscription": sub.name, "error": str(e), "event": raw})
            sub.queue.task_done()


# ──────────────────────── 이벤트 발행자(시작 이벤트) ────────────────────────
class BidProducer(Component):
    """입찰자의 입찰을 BidPlaced 이벤트로 발행만 한다. 응답을 기다리지 않는다."""
    BIDDERS = ["민수", "지영", "철수", "영희", "하준"]

    def __init__(self, broker):
        super().__init__("BidProducer", broker)
        self.rng = random.Random(SEED)

    def produce(self, auction_id: str, count: int, short_ttl_idx=()) -> float:
        """입찰 count건을 모두 발행하고 걸린 시간(ms)을 돌려준다."""
        amounts = [self.rng.randrange(100, 400) for _ in range(count)]
        for i in (6, 15):                           # 13의 배수 입찰을 반드시 포함(분석 실패 시연)
            amounts[i] = (amounts[i] // 13 + 1) * 13
        t = time.perf_counter()
        for i, amt in enumerate(amounts):
            bidder = self.rng.choice(self.BIDDERS)
            # 일부러 ttl=0 을 줘서 만료 폐기를 시연한다(발행 즉시 만료로 간주).
            ttl = 0 if i in short_ttl_idx else 5000
            self.broker.publish("bid.placed", "BidPlaced",
                                {"auction_id": auction_id, "bidder": bidder, "amount": amt}, ttl)
        elapsed = (time.perf_counter() - t) * 1000
        log(self.name, f"입찰 {count}건 발행 완료 ({elapsed:.3f}ms, 처리 결과는 기다리지 않음)")
        return elapsed


class AuctionClock(Component):
    """마감 시각이 되면 AuctionClosed 를 발행한다."""
    def close(self, auction_id: str) -> None:
        log(self.name, f"{auction_id} 경매 마감 -> AuctionClosed 발행")
        self.broker.publish("auction.closed", "AuctionClosed", {"auction_id": auction_id})


# ──────────────────────── 이벤트 구독자(프로세서) ────────────────────────
class BidCapture(Component):
    """모든 입찰을 기록한다."""
    def __init__(self, broker):
        super().__init__("BidCapture", broker)
        self.records: list[dict] = []
        self.listen("bid.placed", self.handle)

    async def handle(self, env, wid):
        self.records.append(env.payload)
        log(wid, f"입찰 기록 {env.payload['bidder']} {env.payload['amount']}원")


class BidTracker(Component):
    """최고가를 추적하고 바뀌면 HighestBidChanged 를 발행한다."""
    def __init__(self, broker):
        super().__init__("BidTracker", broker)
        self.highest: dict[str, dict] = {}
        self.listen("bid.placed", self.handle)

    async def handle(self, env, wid):
        p = env.payload
        cur = self.highest.get(p["auction_id"])
```

```
        if cur is None or p["amount"] > cur["amount"]:
            self.highest[p["auction_id"]] = p
            log(wid, f"최고가 갱신 {p['bidder']} {p['amount']}원 -> HighestBidChanged 발행")
            self.broker.publish("bid.highest_changed", "HighestBidChanged", dict(p))


class BidAnalytics(Component):
    """느린 통계(입찰 수·평균가). 13의 배수 금액에서 일부러 실패한다. 워커 2개로 확장."""
    def __init__(self, broker):
        super().__init__("BidAnalytics", broker)
        self.count = 0
        self.total = 0
        self.first_start = self.last_end = None
        self.worker_use: dict[str, int] = {}
        self.listen("bid.placed", self.handle, workers=2)

    async def handle(self, env, wid):
        if self.first_start is None:
            self.first_start = time.perf_counter()
        await asyncio.sleep(0.02)                    # 느린 처리 흉내
        amt = env.payload["amount"]
        if amt % 13 == 0:                            # 내고장성 시연: 의도적 실패
            raise ValueError(f"{amt}원은 13의 배수")
        self.count += 1
        self.total += amt
        self.worker_use[wid] = self.worker_use.get(wid, 0) + 1
        self.last_end = time.perf_counter()
        log(wid, f"통계 갱신 입찰수={self.count} 평균={self.total / self.count:.1f}원")


class Notifier(Component):
    """최고가 변경·낙찰 이벤트를 받아 알림을 보내고 NotificationSent 를 발행한다."""
    def __init__(self, broker):
        super().__init__("Notifier", broker)
        self.sent = 0
        self.listen("bid.highest_changed", self.on_highest, sub_name="Notifier/highest")
        self.listen("auction.settled", self.on_settled, sub_name="Notifier/settled")

    def _send(self, wid, text):
        self.sent += 1
        log(wid, f"알림 발송: {text}")
        self.broker.publish("notification.sent", "NotificationSent", {"text": text})

    async def on_highest(self, env, wid):
        self._send(wid, f"새 최고가 {env.payload['amount']}원 ({env.payload['bidder']})")

    async def on_settled(self, env, wid):
        p = env.payload
        self._send(wid, f"낙찰 결과: {p['winner']} 님 {p['price']}원")


class Settlement(Component):
    """마감 시 낙찰자를 확정한다. 최고가는 구독한 이벤트로 스스로 유지한다(BidTracker 호출 없음)."""
    def __init__(self, broker):
        super().__init__("Settlement", broker)
        self.my_highest: dict[str, dict] = {}
        self.result = None
        self.listen("bid.highest_changed", self.on_highest, sub_name="Settlement/highest")
        self.listen("auction.closed", self.on_closed, sub_name="Settlement/closed")

    async def on_highest(self, env, wid):
        self.my_highest[env.payload["auction_id"]] = env.payload

    async def on_closed(self, env, wid):
        aid = env.payload["auction_id"]
        top = self.my_highest.get(aid)
        if top is None:
            log(wid, f"{aid} 입찰 없음 - 유찰")
            return
        self.result = {"auction_id": aid, "winner": top["bidder"], "price": top["amount"]}
        log(wid, f"낙찰 확정 {top['bidder']} {top['amount']}원 -> AuctionSettled 발행")
```

```
        self.broker.publish("auction.settled", "AuctionSettled", dict(self.result))


class DeadLetterMonitor(Component):
    """deadletter 토픽의 이벤트를 기록한다."""
    def __init__(self, broker):
        super().__init__("DeadLetterMonitor", broker)
        self.items: list[dict] = []
        self.listen("deadletter", self.handle)

    async def handle(self, env, wid):
        self.items.append(env.payload)
        log(wid, f"DLQ 수신: {env.payload['subscription']} / {env.payload['error']}")


# ─────────────────────── 제약 검사 ───────────────────────
def find_component_refs(obj, depth=0) -> list:
    """객체 안에서 Component 참조(직접·컨테이너·바운드 메서드)를 찾는다."""
    if depth > 4:
        return []
    if isinstance(obj, Component):
        return [obj.name]
    if hasattr(obj, "__self__") and isinstance(getattr(obj, "__self__"), Component):
        return [obj.__self__.name]
    items = []
    if isinstance(obj, dict):
        items = list(obj.keys()) + list(obj.values())
    elif isinstance(obj, (list, tuple, set)):
        items = list(obj)
    out = []
    for it in items:
        out += find_component_refs(it, depth + 1)
    return out


def check_c1(components) -> bool:
    """C1: 컴포넌트 속성에 다른 컴포넌트 참조가 없고, 브로커만 참조함을 확인."""
    ok = True
    for c in components:
        bad, n = [], 0
        for k, v in vars(c).items():
            n += 1
            refs = [r for r in find_component_refs(v) if r != c.name]
            if refs:
                bad.append((k, refs))
        has_broker = isinstance(vars(c).get("broker"), EventBroker)
        status = "OK" if not bad and has_broker else f"위반 {bad}"
        ok &= status == "OK"
        log("제약검사C1", f"{c.name:<17} 속성 {n}개 검사, 타 컴포넌트 참조 {sum(len(r) for _, r in bad)}, 브로커 참조
{'있음' if has_broker else '없음'} -> {status}")
    # 자가검증: 검사기가 위반을 실제로 잡아내는지 확인(가짜 위반 객체 사용)
    a, b = Component("A", None), Component("B", None)
    a.peer = b
    caught = any(r != "A" for v in vars(a).values() for r in find_component_refs(v))
    log("제약검사C1", f"검사기 자가검증(A가 B를 참조하게 만든 가짜 객체) -> 위반 탐지 {'성공' if caught else '실패'}")
    ok &= caught
    log("제약검사C1", "결과: " + ("PASS" if ok else "FAIL"))
    return ok


def check_c3_c4(broker, producers) -> tuple[bool, bool]:
    """C3: 모든 이벤트 type 이 과거형(-ed). C4: 발행자는 구독/구독자 정보를 갖지 않음."""
    c3 = all(t.endswith(("ed", "Sent")) for t in broker.types_seen)  # Sent 도 과거형
    c4 = all(not any(isinstance(v, Subscription) for v in vars(p).values()) and not hasattr(p, "subs_of")
             for p in producers)
    log("제약검사C3", f"이벤트 type {sorted(broker.types_seen)} -> {'PASS' if c3 else 'FAIL'}")
    log("제약검사C4", f"발행자는 토픽 이름만 사용, 구독자 목록 미보유 -> {'PASS' if c4 else 'FAIL'}")
    return c3, c4


# ─────────────────────── 실행 ───────────────────────
```

```python
async def main() -> int:
    broker = EventBroker()
    producer, clock = BidProducer(broker), AuctionClock("AuctionClock", broker)
    capture, tracker = BidCapture(broker), BidTracker(broker)
    analytics, notifier = BidAnalytics(broker), Notifier(broker)
    settle, dlq_mon = Settlement(broker), DeadLetterMonitor(broker)
    subscribers = [capture, tracker, analytics, notifier, settle, dlq_mon]
    allc = [producer, clock] + subscribers
    log("시스템", "경매 시스템 시작")
    c1 = check_c1(allc)

    pub_ms = producer.produce("A-100", 26, short_ttl_idx=(3, 11, 20))
    await broker.drain()                        # 입찰 처리가 모두 끝나면 마감(시연 순서 보장)
    clock.close("A-100")
    await broker.drain()
    c2_total = (analytics.last_end - analytics.first_start) * 1000
    log("제약검사C2", f"입찰 전체 발행 {pub_ms:.3f}ms vs BidAnalytics 처리 {c2_total:.1f}ms "
                      f"(약 {c2_total / max(pub_ms, 1e-6):.0f}배) -> 발행자는 처리 완료를 기다리지 않음")
    c3, c4 = check_c3_c4(broker, [producer, clock])

    workers = {}
    for c in subscribers:
        workers.update(c.workers)
    broker.close(workers)
    await asyncio.gather(*(t for c in subscribers for t in c.tasks))

    # 요약 표
    out = ["", "=" * 70, "요약", "=" * 70, "[토픽별 발행 수]"]
    out += [f"  {t:<22}{n:>4}" for t, n in broker.published.items()]
    out += ["", "[구독별 처리/재시도/DLQ/만료]", f"  {'구독':<24}{'처리':>5}{'재시도':>6}{'DLQ':>5}{'만료':>5}"]
    for ss in broker.subs.values():
        out += [f"  {s.name:<24}{s.processed:>5}{s.retried:>7}{s.dlq:>5}{s.expired:>5}" for s in ss]
    out += [f"[BidAnalytics 워커별 성공 처리] {dict(sorted(analytics.worker_use.items()))}"]
    r = settle.result
    out += [f"[낙찰 결과] {r['auction_id']} 낙찰자={r['winner']} 낙찰가={r['price']}원 (알림 {notifier.sent}건 발송)"
if r else "[낙찰 결과] 유찰"]
    out += [f"[BidTracker 최고가] {tracker.highest['A-100']['amount']}원 / Settlement 보유 {r['price'] if r else '-'}
원 -> 일치 {r and r['price'] == tracker.highest['A-100']['amount']}"]
    out += ["", "[제약 검사]",
            f"  C1 직접 참조 없음   : {'PASS' if c1 else 'FAIL'}",
            f"  C2 비동기 발행      : 발행 {pub_ms:.3f}ms < 분석 {c2_total:.1f}ms -> {'PASS' if pub_ms < c2_total
else 'FAIL'}",
            f"  C3 과거형 이벤트    : {'PASS' if c3 else 'FAIL'}",
            f"  C4 발행자 익명성    : {'PASS' if c4 else 'FAIL'}"]
    for line in out:
        LOG.append(line)
        print(line)
    return 0


if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8")
    code = asyncio.run(main())
    path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "run_log.txt")
    with open(path, "w", encoding="utf-8") as f:
        f.write("\n".join(LOG) + "\n")
    sys.exit(code)
```